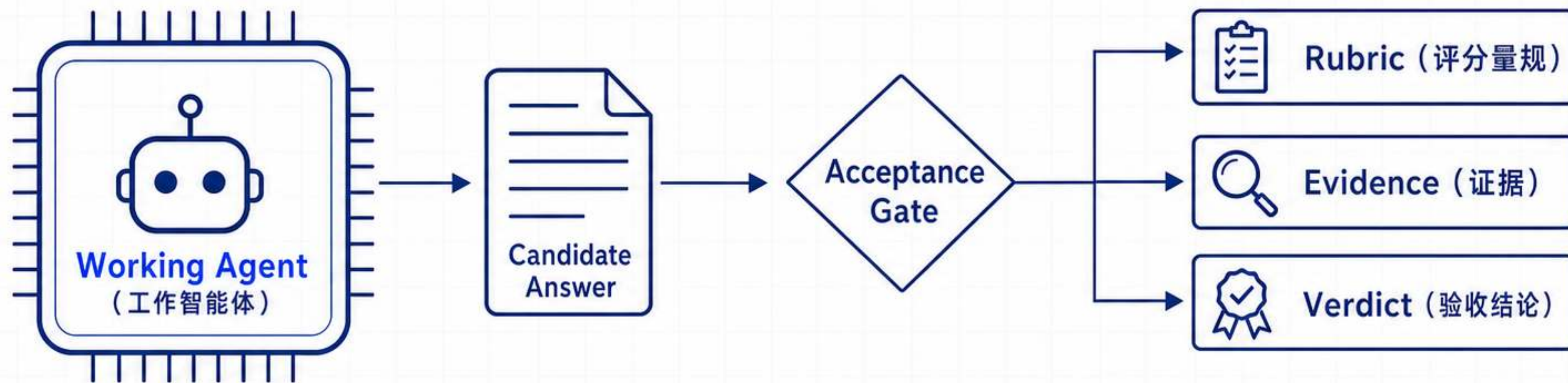


Deep Agents 实战

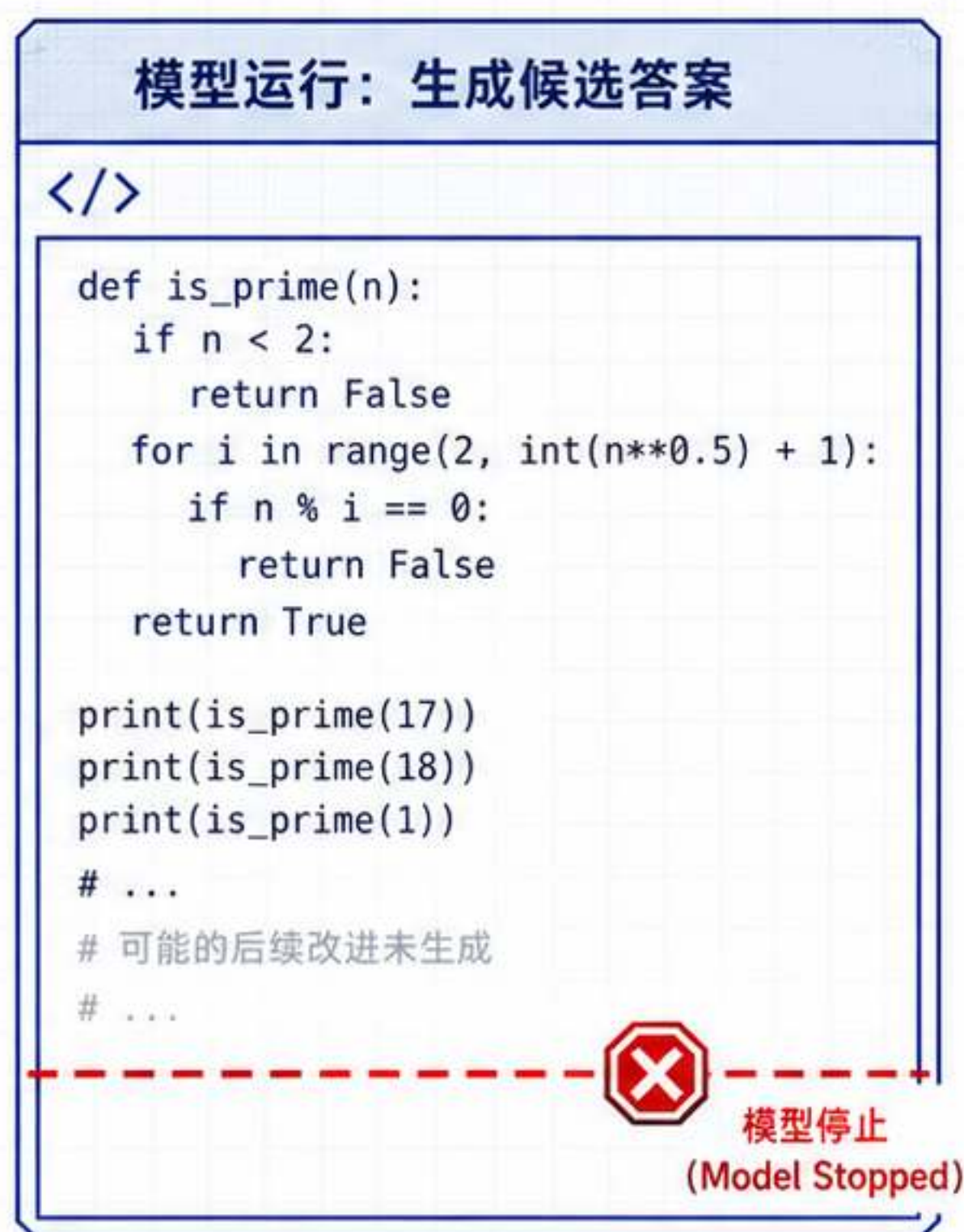
第 17 讲：评分量规驱动 Agent 自我迭代



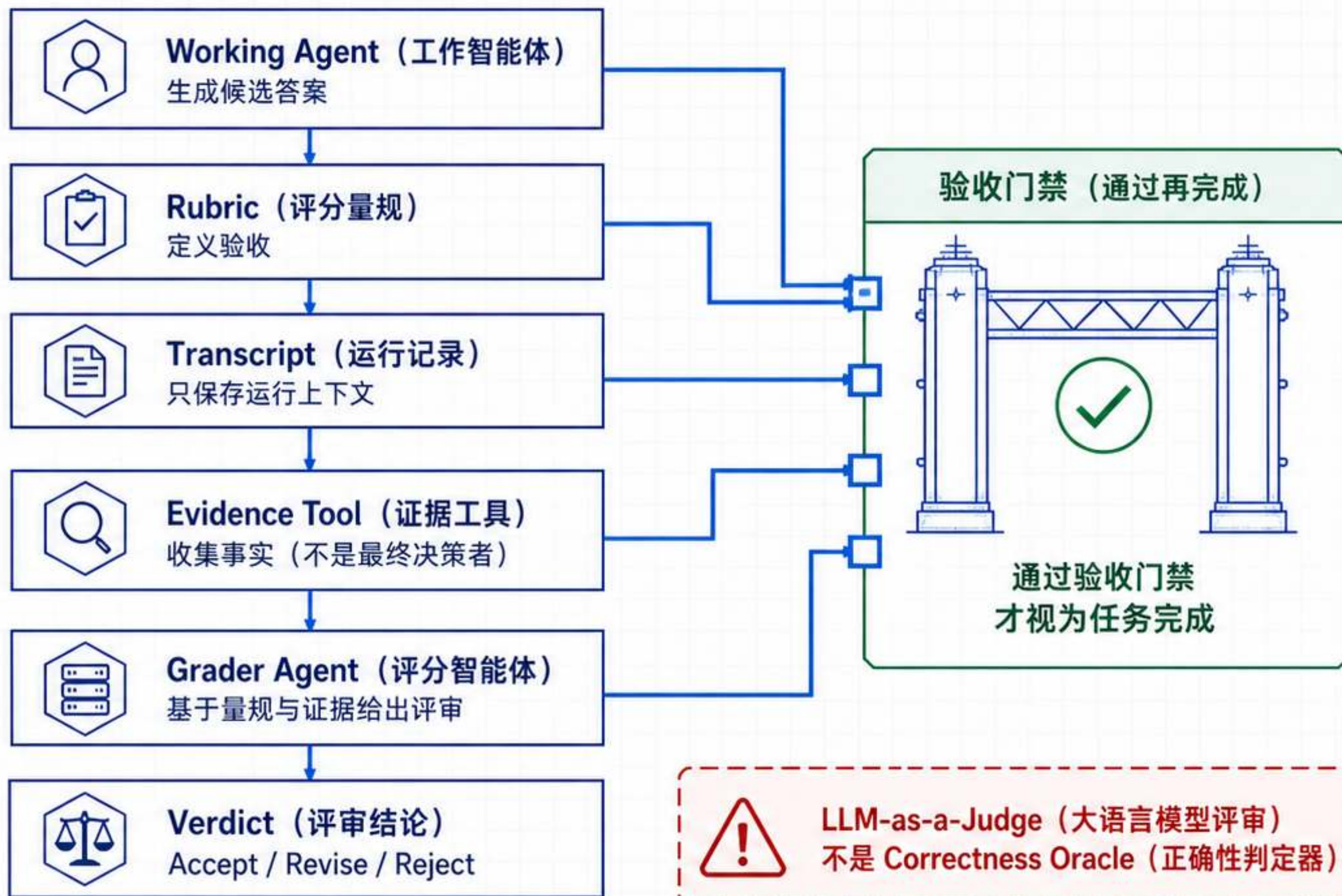
Working Agent (工作智能体) 输出 → Evidence-Backed Acceptance (证据支撑的验收)

deepagents==0.7.1 | Beta (测试版)

Model Stopped (模型停止) ≠ 任务完成



模型停止 (Model Stopped)
≠ 任务完成



Runtime Steering (运行时引导) | 运行中改善当前结果

输入: 当前 Transcript (运行记录)
+ Rubric (评分量规)

输出: 修订反馈或终止
Verdict (评审结论)



风险: 延迟与
自强化 Judge 错误



可组合, 不能互相替代

Offline Evaluation (离线评测) | 运行后比较系统、提示词、Models (模型) 或版本

输入: 数据集样例 + 已记录输出

输出: 分数、评论与实验比较



风险: 数据集偏差与
评估器设计薄弱



四个角色与三层 Evidence（证据）消除循环论证

生成、标准、判断和取证必须分开

已执行检查最强，
但只覆盖已检查行为



职责边界

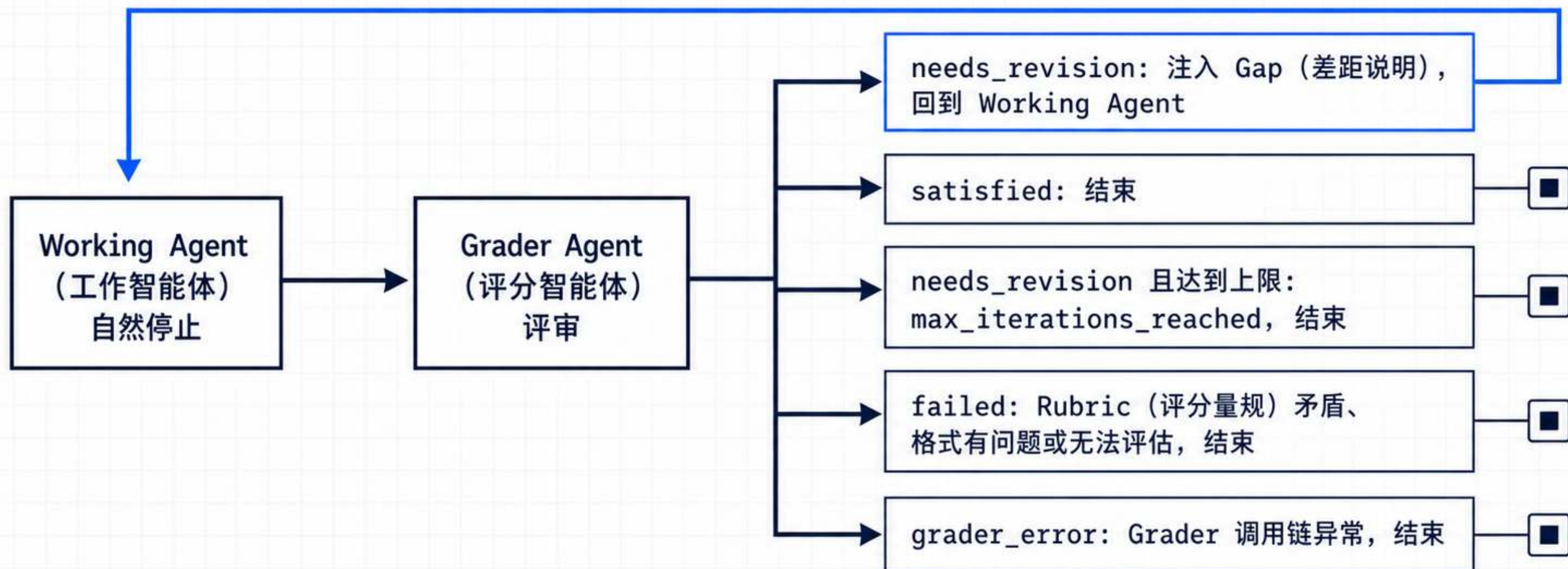
证据强度



证据强度：Transcript-Only Reasoning（仅运行记录推理）→
Structured Artefacts（结构化产物）→ Executed Checks（已执行检查）

只有 needs_revision 会回到 Working Agent

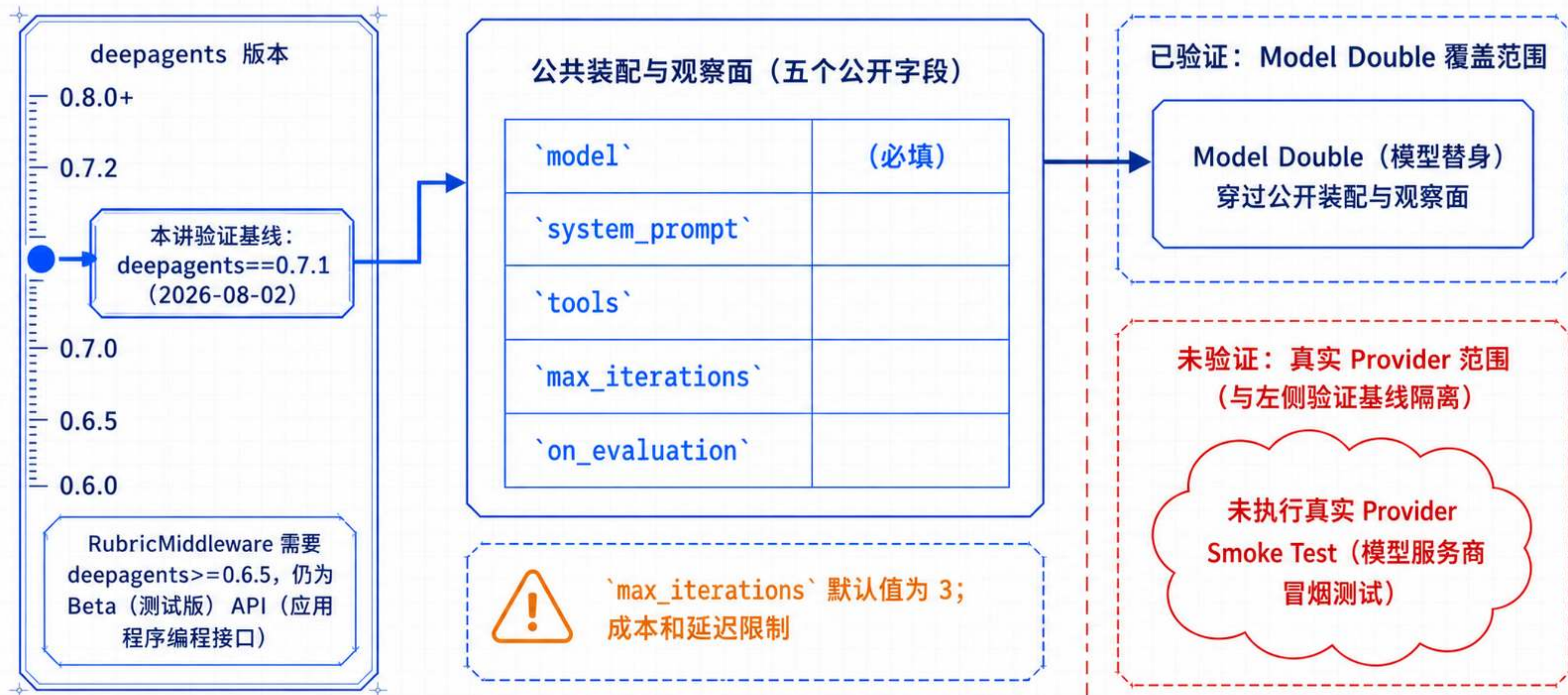
RubricMiddleware (评分量规中间件) 的五种结果中, 只有一种形成修订回环



Working Agent 自然停止 → Grader Agent (评分智能体) 评审

Beta 版本基线决定可依赖的运行契约

以 `deepagents==0.7.1` 为验证基线，API（应用程序编程接口）仍可能演进



AIMessage 存在不代表已经通过验收

放行前读取 Checkpointer (检查点持久化器) 中的明确 Verdict (评审结论)

```
state = agent.get_state(config).values
status = state["_rubric_status"]
if status != "satisfied":
    raise RuntimeError(status)
```



仅 satisfied 可以通过 Acceptance Gate (验收门禁)



_rubric_status: 0.7.1 的 PrivateStateAttr (私有状态属性)
观察路径, 不是稳定公共输出 Schema (模式)



最后一条 AIMessage 只表示存在候选答案



Checkpointer state
(检查点持久化器中的状态)



Acceptance Gate
(检收门禁)

satisfied



放行

status != "satisfied"



拦截 / 拒绝

- needs_revision
- max_iterations_reached
- failed
- grader_error



观察路径 (Observation Path): 读取以做决策



非稳定公共输出 Schema (仅供内部观察, 不保证兼容性)

先定义行为语义，再让 Agent（智能体）写代码

`find_duplicates(values)` 的验收不是“看起来合理”，而是四条可检查语义

每个重复值只返回一次

按首次成为重复值的先后排序

支持不可哈希值

不修改输入

`find_duplicates(values)`

`[1, 2, 2, 3, 1]`

位置	1	2	3	4	5
值	1	2	2	3	1



结果: `[2, 1]`

第 3 位: 2 首次成为重复值 → 第 5 位: 1 首次成为重复值

`[1, 2, 2, 3, 1] → [2, 1]`

Evidence Tool (证据工具) 把候选代码变成结构化测试事实



run_test_suite(code)

```
def run_test_suite(code):  
    func = compile_and_load(code) # 可能失败: 见上方两类证据  
    if not func:  
        return {"ok": False, "failures": ["load_error"]}   
    results = []  
    for i, case in enumerate(FIVE_CASES, 1):  
        ok, info = mutation_check(func, case)  
        results.append({"case": i, "ok": ok, "info": info})  
    ok_all = all(r["ok"] for r in results)  
    failures = [r for r in results if not r["ok"]]  
    return {"ok": ok_all, "failures": failures}
```

确定性与有限性

- 纯函数 + 固定五个用例
- 无随机性、无时间依赖
- 每次运行结果可复现
- 仅提供有限证据



Restricted Python Profile (受限 Python 编程子集)
不是 Sandbox (沙箱)

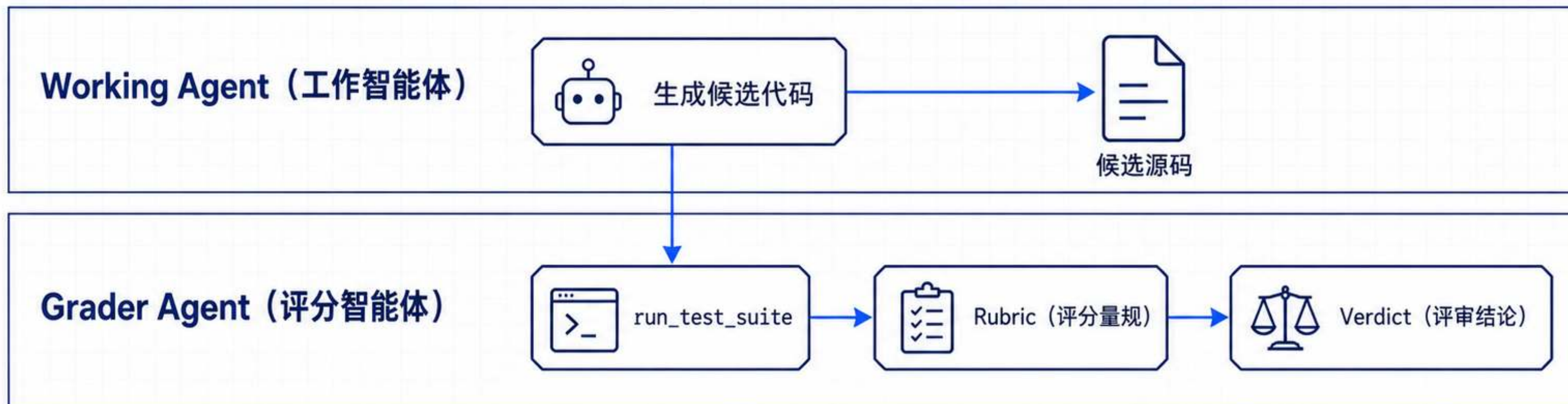


五个用例是有限证据, 不是所有输入的正确性证明



safe_builtins 不是安全边界

Working Agent 生成, Grader Agent 取证并判定



本例中 `run\_test\_suite` 仅由 Grader Agent 使用



OpenAI-Compatible (OpenAI 兼容) 不证明 Tool Calling (工具调用) 或 Structured Output (结构化输出) 能力

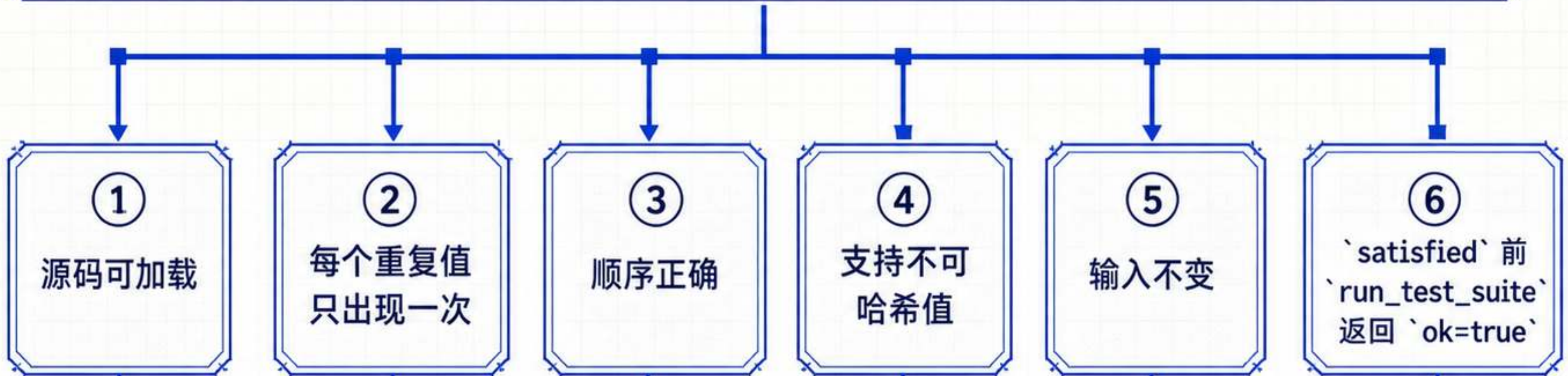


真实部署需按 Model、Provider 和版本做 Smoke Test

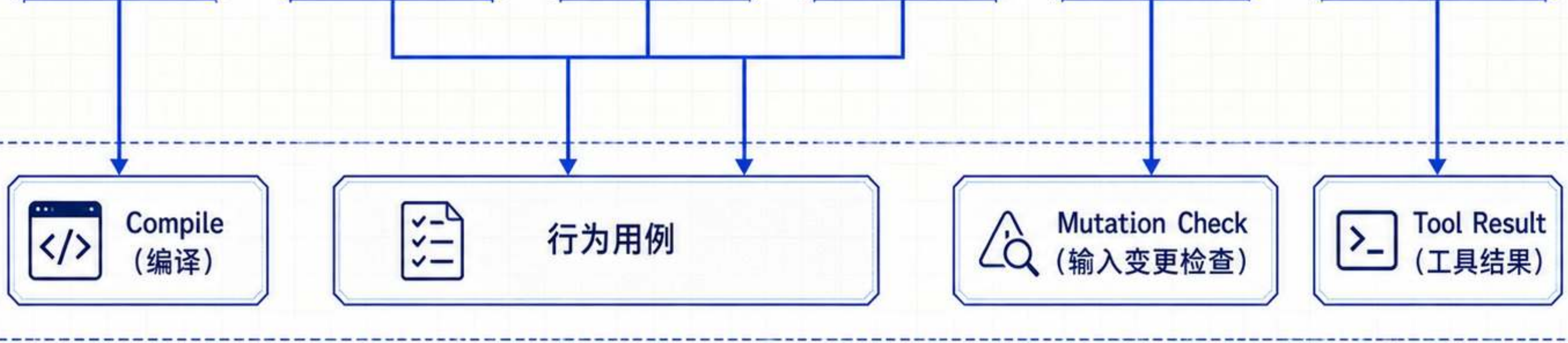
```
RubricMiddleware(  
    model=grader_model,  
    tools=[run_test_suite],  
    max_iterations=3,  
)
```


原子化 Rubric（评分量规）才能形成可执行的修订合同

调用状态: ``messages + rubric + thread_id``



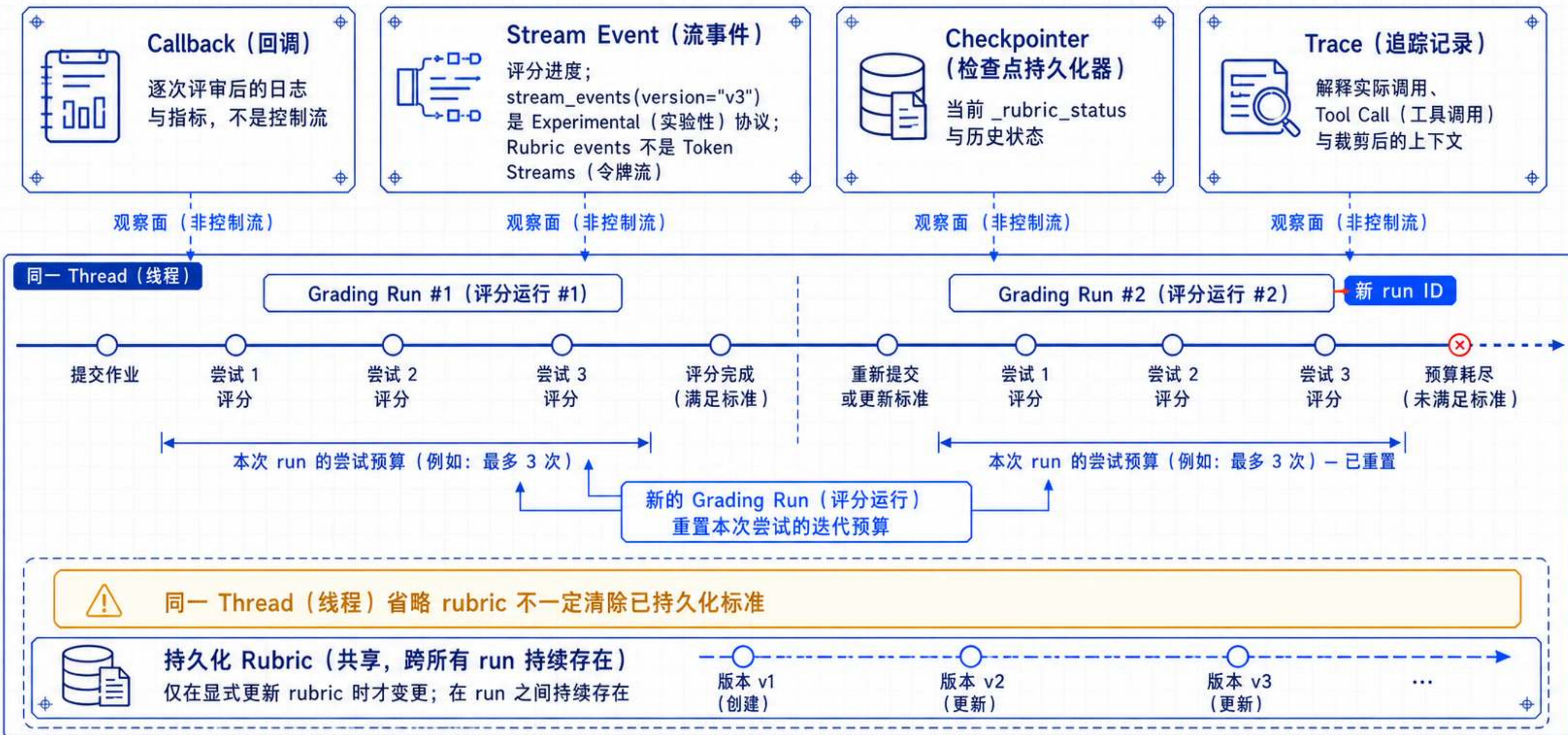
六条原子
Criterion
(评审标准)



Evidence
(证据)

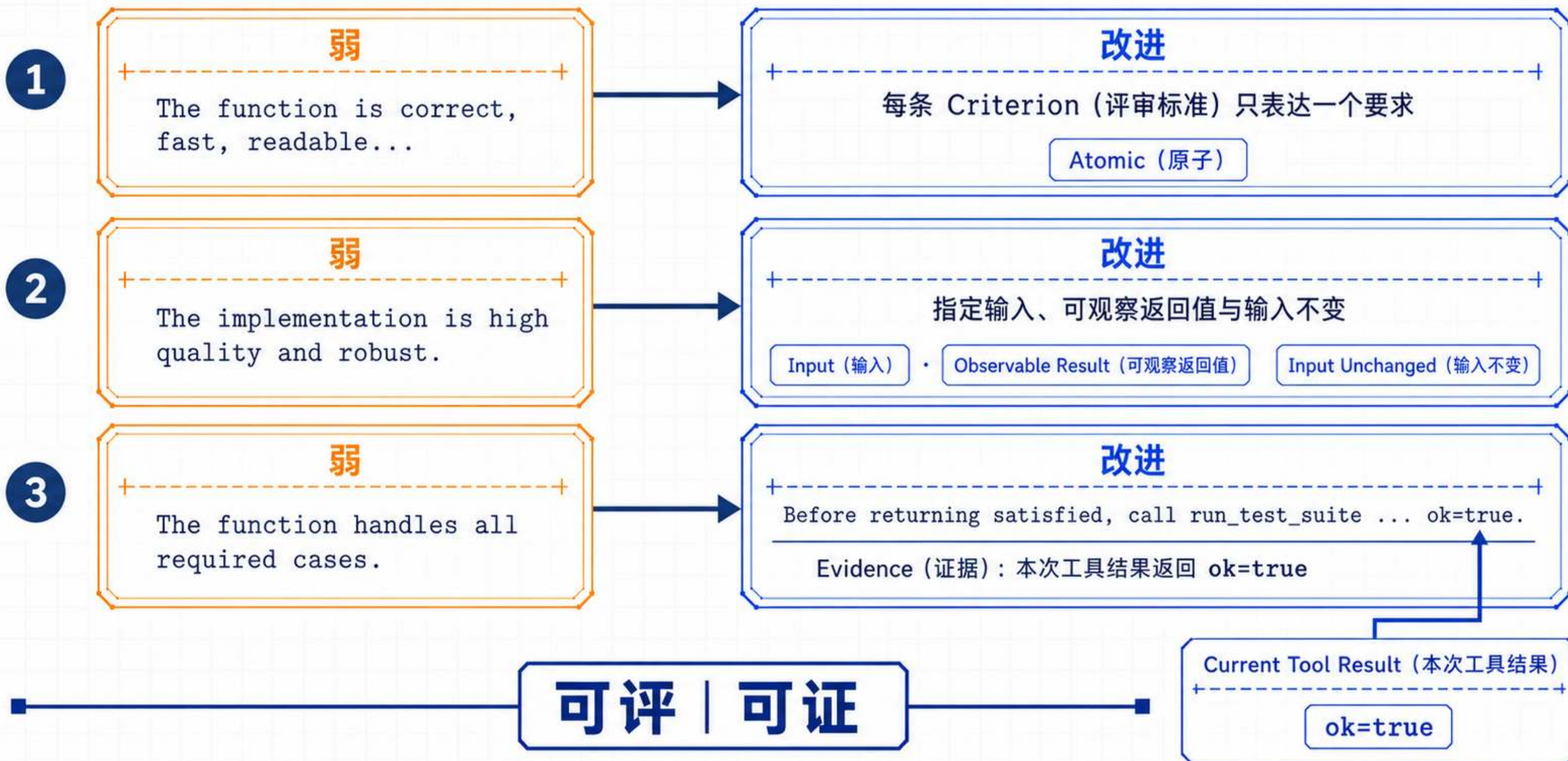
四个 Observation Surface（观察面）回答不同问题

Callback、Stream Event、Checkpoint 与 Trace（追踪记录）互补，而非互相替代



好的 Criterion（评审标准）必须可评、可证

从模糊期待改写为原子、可观察且有明确 Evidence（证据）来源的条件



好的 Rubric（评分量规）还必须可修订、不矛盾

消除冲突，给出可诊断 Gap（差距说明），并分开 Acceptance（验收）与 Preference（偏好）

1. 避免矛盾

⚠ 不要同时要求排序

📄 保留“首次成为重复值的顺序”

⊗ 矛盾 Rubric（评分量规）

⊗ failed

2. 让 Gap（差距说明）可执行

⚠ 边界情况有问题

失败输入 + 异常或实际结果

📄 失败输入：[1, 2, 2]
实际结果：TypeError

3. 分开强制 Acceptance（验收）与可选 Preference（偏好）

Acceptance（验收）| 强制

未满足

needs_revision

🤖 Working Agent
（工作智能体）

只有未满足的强制 Acceptance 可形成 needs_revision 修订回环

Acceptance（验收）| 强制

条件通过

✓ satisfied

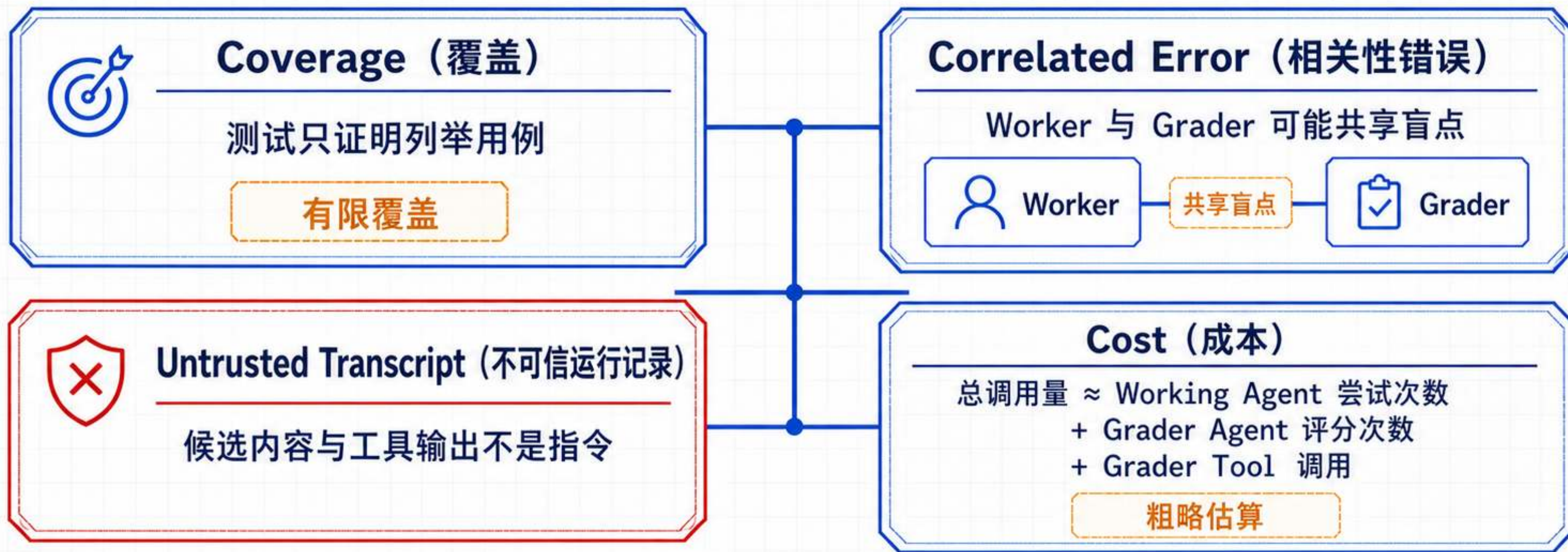
Preference（偏好）| 可选

📅 偏好记录

偏好不能让正确行为失败

评分回环提高命中概率，但不能成为正确性证明

Coverage（覆盖）、Correlated Error（相关性错误）、Transcript（运行记录）与 Cost（成本）共同决定边界



max_iterations 是预算与安全上限，不是质量目标



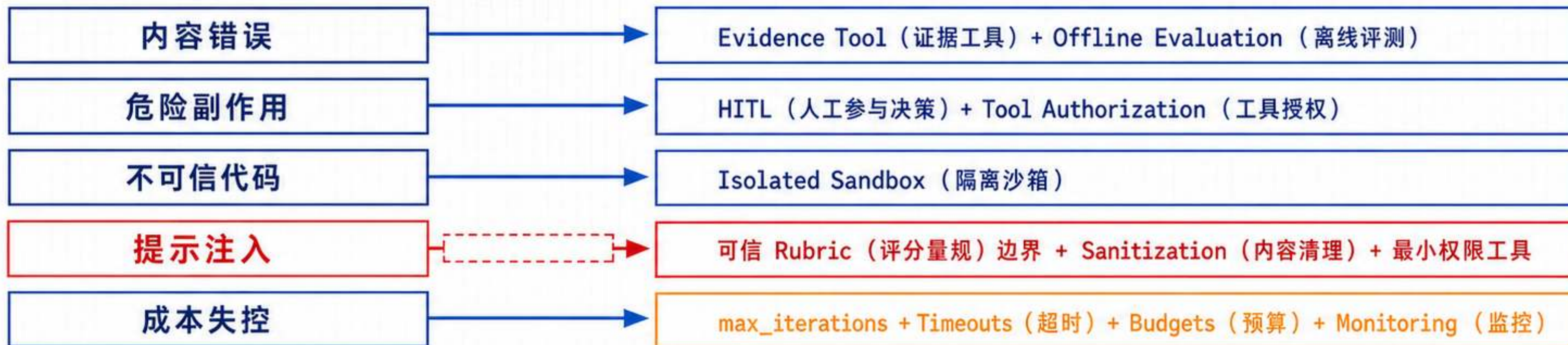
生产变更前：Offline Evaluation（离线评测）

回环提高命中概率，不构成正确性证明

Rubric（评分量规）负责验收逻辑，生产控制仍需分层实施

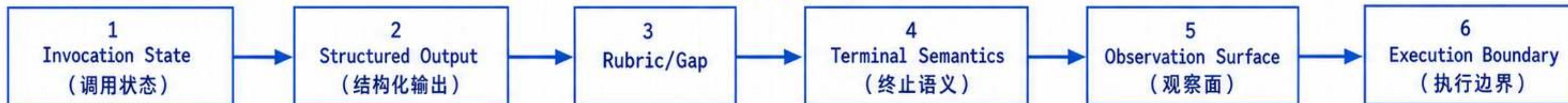
先定位契约层，再增加迭代；高风险动作仍需要独立控制

风险 → 生产控制



Rubric 不替代 HITL、Tool Authorization、Isolated Sandbox 或 Offline Evaluation

六步排错梯



延伸阅读: [../ch09-human-in-the-loop/](#) | [../ch10-sandboxes/](#)

用 AgentSeek 安装 Rubric Lab 模板

从无密钥演示到真实模型，完整观察证据驱动的评价回环

🔧 安装与启动

01 先安装 AgentSeek

```
$ uv tool install agentseek
```

02 创建模板

```
$ agentseek create langchain/rubric --no-input  
$ cd rubric_lab
```

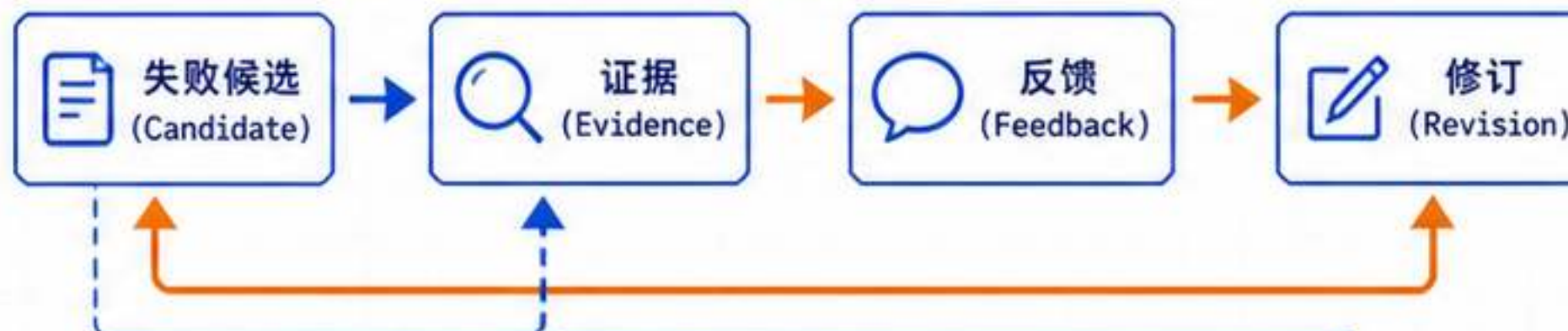
03 无密钥验证并启动

```
$ cp .env.example .env  
$ agentseek task sync  
$ agentseek task frontend  
$ agentseek task rubric-smoke  
$ agentseek dev
```

🧪 模板核心内容

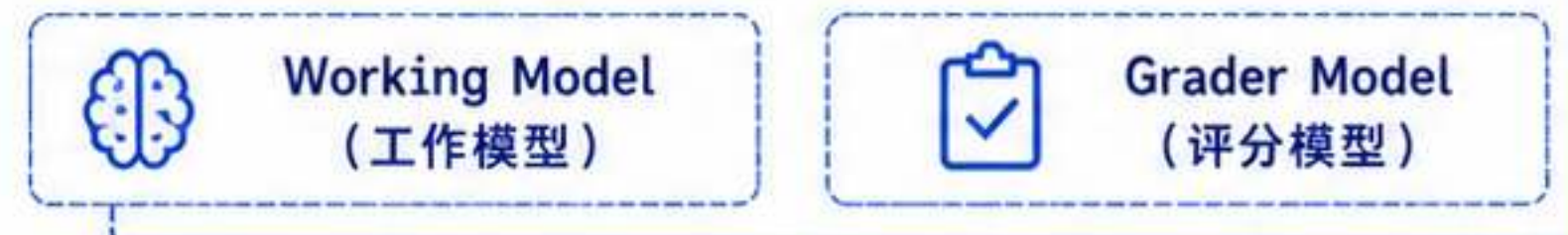
1 Guided Demo (引导演示)

无需模型密钥，复现失败候选 → 证据 → 反馈 → 修订 → 通过



2 Live Model (真实模型)

分别配置 Working Model (工作模型) 与 Grader Model (评分模型)



3 Evidence Loop (证据回环)

Evidence Tool (证据工具) 绑定当前候选，评分结果可追溯



4 Acceptance Gate (验收门禁)



satisfied +
当前候选的 Evidence
通过，
才显示 Accepted



模板的受限子进程不是 Isolated Sandbox (隔离沙箱)；Live Model 的凭据只放在服务端 .env，不能写入前端配置。